

Is Bravo actually faster to build in?

“Developing in Lora is hard, but developing in Bravo is faster and easier because the team now knows what to do.”

0 ISSUES ASSIGNED TO ANYONE ON THE TWO BOARDS THE CLAIM POINTS AT	1.27_x MORE TICKETS FROM BRAVO THAN LORA, JAN-AUG. THIS REVERSED FROM 1.83_x THE OTHER WAY ONCE WE COUNTED THE MISSING LN PROJECT.	2_d MEDIAN TIME TO CLOSE A TICKET IN AUGUST – THE SAME ON BOTH PLATFORMS	5→1 REPOSITORIES TOUCHED PER CHANGE. THIS IS THE REAL EFFECT.
---	--	--	---

Six claims, checked against the data

The DF4W and DF2W boards show Bravo development moving faster	● NOTHING HERE TO M	DF holds 24 issues, 23 of them Epics. D2W holds 22, all Epics. Not one is assigned to anyone, and not one is resolved. These are product intake backlogs. You cannot time anything on them.
Bravo development is faster	● NOT SUPPORTED	Bravo's two delivery projects (BLCS and LN) created about 6,518 issues between January and August, against LORA's 5,144 — so Bravo is ahead, 1.27× . That <i>reverses</i> our earlier 1.83× finding for LORA, which counted only BLCS . But a ticket count measures how finely work is split up as much as how much gets done, so this retires the measure rather than supporting the claim. In August the median time from created to resolved is 2 days on both . In June LORA was <i>faster</i> .
...because the team now knows what to do	● TENURE EXPLAINS I	Bravo's team has worked on this codebase since 2022 — 94 authors all-time, 23 of them active in 2026. That is long tenure on a four-year-old codebase. It is not a platform being easier to learn.
Developing in Bravo is <i>easier</i>	● SUPPORTED	One change produces fewer pieces of work. A Bravo ticket splits into [BE] , [FE] and [QA] . The same change in LORA splits into five repositories and five deploys. That is real, it is measurable, and it is the defensible core of the claim.
Bravo is easier to join	● HALF TRUE	True on skills: Java, Spring and Camunda are common and easy to hire for. Not true on the codebase: a new hire has to read a 498,000-line monolith in which the product is identified in five different places. The advantage is used up by week three.
DF2W is a live Bravo product delivering faster than LORA	● REFUTED	Corrected 2026-09-10: DF2W is not live at all. It is in UAT, with a penetration test running against the LOS, and its go-live epics are still open. This pack had called it “fully configured in production”, which reads as released but unused. It is pre-release. It has started zero applications in 90 days. DF4W carries 5.5% of Bravo's volume. The two products the claim cites are the smallest things Bravo runs — and one of them is not running at all.

The claim is about what it is like **to work in a platform**, and it is being defended with **how fast tickets close**.

Ticket speed cannot settle it. The measure reversed direction the moment we added a project we had missed, which is the clearest possible sign it was never measuring what it was asked to measure. On what it is like to work in, the claim is right — and until now nobody had measured how big the effect is: **five repositories per change, against one or two**.

WHAT THE DATA WILL NOT SUPPORT

No throughput measure here can be trusted, in either direction. The ticket ratio *reversed* when we added a missing Bravo project: LORA ahead 1.83× became Bravo ahead 1.27×. A measure that swings that far on a scope fix is not measuring delivery. On cycle time, the August medians tie at **1 to 2 days across all three projects**. June’s apparent gap in Bravo’s favour comes from batching releases and loading a backlog, not from architecture.

WHAT THE DATA CONFIRMS

One integration change touches **1 or 2 repositories and takes 1 or 2 deploys** in Bravo — two when a console is involved. The same change in LORA touches **5 repositories and takes 5 deploys**, in a required order. That is the whole real effect, and it survives the scope correction: two is still not five.

The two boards the claim points at contain no development work

...but Bravo has **two boards that do** — **BLCS** (Team Scoring & Underwriting) and **LN** (Team Surveyor & Verificator). The claim points at the wrong boards. It is not that there is no work; see the next slide.

PROJECT	BOARD	ISSUES	TYPES	ASSIGNED	RESOLVED	FIRST → LAST CREATED
DF (DF4W)	2099, kanban	24	23 Epic, 1 Task	0	0	2025-12-12 → 2026-09-09
D2W (DF2W)	2877, kanban	22 of 36 keys	22 Epic	0	0	2026-06-02 → 2026-08-28
BL (LORA DP)	683	10,138 keys	Story, Sub-task, Task, Bug	assigned	thousands	2024-11-21 → 2026-09-10

Twelve of the 24 **DF** issues were created inside **four minutes** on 2025-12-12. That is somebody loading a backlog, not a sprint. The first sixteen **D2W** issues were created inside **seventeen minutes**. Nine months after the first DF epic was written, not one has been assigned to anyone, and not one has moved out of To Do.

THIS IS NOT A CRITICISM OF THE BOARDS

A product intake backlog is a perfectly good thing to have, and both of these are well made. The point is what you can conclude from them. **A timeline view of these boards shows what product intends to build, not what has been delivered.** Putting board 2099 next to board 683 compares a backlog of epics with a stream of delivered tickets.

Bravo’s development work lives in BLCS and LN

The comparison the claim wants has to be made somewhere else — and there are **two** such places, not one. Recent BLCS titles: “[BE] Prepare Query Change GMB to NMH”, “[DF2W UW] Sync Surveyor Status Underwriting Return”. Recent LN titles: “[Surveyor] Revamp Filter Assignment Team Head Surveyor”, “[FE][TechDebt] Operation Route Validation based on Role”. We added LN on 2026-09-10, after Team Bravo pointed out it was missing. It is the larger of the two.

	BLCS — BRAVO LOS	LN — BRAVO SURVEYOR & VERIFICATOR	BL — LORA DP
First issue	BLCS-985 , 2023-12-19	LN-76 , 2024-09-09	BL-3 , 2024-11-21
Latest when we read it	BLCS-4811 , 2026-09-09	LN-6626 , 2026-09-10 created 14:35, resolved 17:43	BL-10138 , 2026-09-10
Issue types in use	Epic, Story, Story Bug, Task, Sub-task, Bug	Epic, Story, Task, Sub-task	Epic, Story, Story Bug, Task, Sub-task, Bug
How sub-tasks are labelled	[BE] [FE] [QA] [DF2W UW]	[Surveyor] [Operation] [FE] [FE][TechDebt] [Master]	[LSS] [LGS] [LPW] [LTW] [LTS] [LB0FE] [QA] [PR]
Delivers into	bravo-bpm-service	the three operator consoles — 763,861 lines of code, 829 tests	13 LORA repositories

The two projects label their work the same way, which is what makes the comparison possible at all — both run the same PMO template. Counting a BLCS ticket against a BL ticket is comparing like with like at the level of *process*. It is **not** like with like at the level of a *change*, which is what §6 is about.

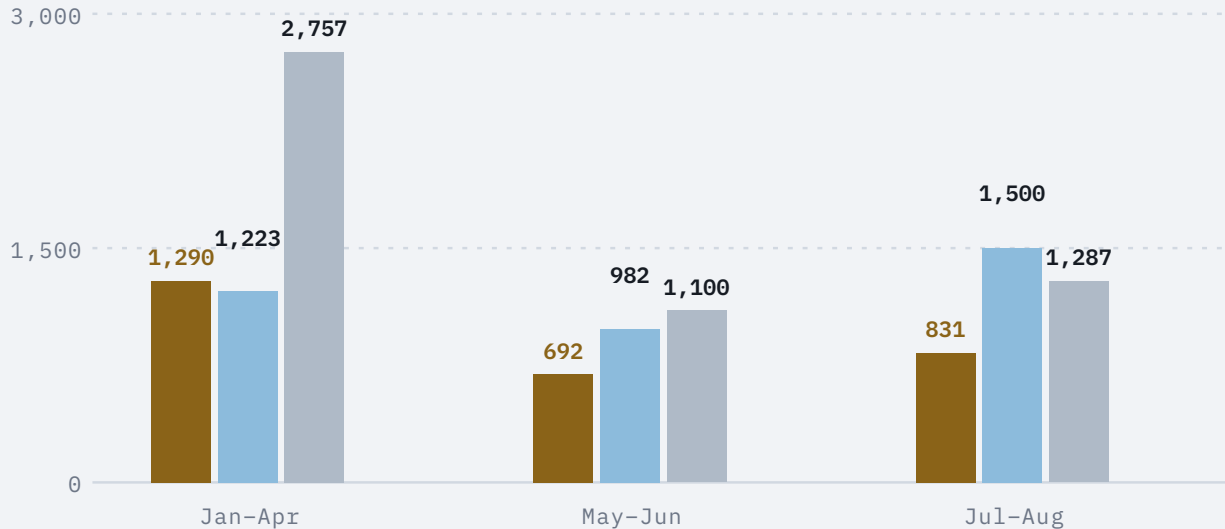
DF2W WORK APPEARS IN BOTH DELIVERY PROJECTS

BLCS-4420 “[DF2W Scoring] Penyesuaian sumber Data Downpayment” and BLCS-4426 sit in one; LN-6479 “[DF2W] E2E Testing” and LN-4533 “[DF2W] [O] Application Tracking” sit in the other. So DF2W **is** being actively built and tested, across two squads. It is simply not tracked on the DF2W intake board.

Its real status, verified 2026-09-10: UAT branch mapping seeded (INS-6245), go-live worker TAC closed (TDF-4273 , 09-09), penetration test running against the LOS (ADI-1312 , BLCS-4799), pre-go-live epics still open (D2W-5 , LN-4573). **Pre-release, not abandoned.**

Ticket volume: once LN is counted, LORA's lead reverses

■ BRAVO · BLCS ■ BRAVO · LN (ADDED 2026-09-10) ■ LORA · BL



Corrected 2026-09-10. With Bravo's second delivery project LN counted — 3,705 issues, more than BLCS — the January to August totals are Bravo 6,518 and LORA 5,144. That is **Bravo ahead 1.27×**, and **1.81×** in July and August. What this slide said before — LORA ahead 1.83×, its lead narrowing from 3.2× to 1.5× — came from counting one of Bravo's two projects against all of LORA's.

METHOD

Jira hands out issue numbers in order within a project, so the first number after a month boundary tells you how many were created before it. We read the boundaries directly: BLCS-1939, LN-2843 and BL-4795 at 1 January, through BLCS-4752, LN-6548 and BL-9939 at 1 September. We could only read LN at four boundaries, so we pooled the windows two months at a time, and pooled the other two projects the same way.

WE TESTED FOR DOUBLE-COUNTING ON 2026-09-10. THE TWO PROJECTS DO NOT OVERLAP.

If a single change were a BLCS back-end story *and* an LN console story, adding the projects together would count it twice. Four checks say that does not happen. (1) BLCS is *Team Scoring & Underwriting* and LN is *Team Surveyor & Verificator* — they are split by **squad and business area, not by tier**. (2) LN sub-tasks are 18 [BE], 15 [FE] and 7 [QA] — full-stack work, not the front-end half of something. (3) The capability prefixes **do not overlap**: [DF2W Scoring] and [UW 4W] against [DF2W] [0] and [DF2W] [S]. (4) There are **zero links** between issues in the two projects. **Of 80 sampled summaries, no pair describes the same change.** So the range collapses to its top end: 6,518, or Bravo ahead 1.27×. What inflation is left comes from per-sprint QA tickets written once per squad — a handful, not a doubling. The number gaps remain an **upper bound** for all three projects, since D2W has 14 of its 36 numbers unused.

WHAT THIS DOES NOT MEAN — AND THE CAUTION NOW CUTS BRAVO'S WAY

It does not mean Bravo ships 1.27 times as much product change. A ticket count measures **how finely work is split up** at least as much as how much gets done. We used that argument to discount LORA's apparent lead. **We now have to use it to discount Bravo's.** The right conclusion is that ticket volume cannot settle this either way. See §6.

Time to close a ticket: no lasting advantage either way

Created date to resolutiondate , in calendar days, for every resolved issue created in a matching window. Two windows, six samples, n = 50 each, taken six months apart so we are not reading a single sprint. Until 2026-09-10 this section sampled only BLCS and BL . LN had been added to §3 and §5 but not here. It is now measured, and the finding gets stronger.

SAMPLE	WINDOW	N	MEDIAN	MEAN	SLOWEST TAIL
Bravo BLCS	2026-06-01 → 06-08	50	9 d	22.3 d	60-98 d (9 issues)
Bravo LN	2026-06-01 → 06-02	50	68 d	63.4 d	40-101 d (45 issues) – a backlog load, not a working rhythm
LORA BL	2026-06-02 → 06-03	50	2 d	18.5 d	43-90 d (13 issues)
Bravo BLCS	2026-08-11 → 08-18	50	2 d	3.9 d	10-15 d (8 issues)
Bravo LN	2026-08-11 → 08-18	50	1 d	2.4 d	8-10 d (9 issues)
LORA BL	2026-08-11 → 08-12	50	2 d	5.3 d	14-29 d (9 issues)

1 • IN AUGUST THE MEDIANS ARE 1 TO 2 DAYS EVERYWHERE

All three projects are indistinguishable — LN 1 day, BLCS 2 days, BL 2 days. And LN , Bravo’s larger project, has both the lowest median and the lowest mean. Whatever difficulty LORA imposes does not show up here. Adding Bravo’s missing project does not open a gap in Bravo’s favour either.

2 • BOTH JUNE FIGURES ARE BACKLOG LOADS, NOT WORKING RHYTHMS

LN ’s 68 days comes from 45 of its 50 issues being created in a 37-minute window on 2026-06-01, between 21:19 and 21:56 — the [DF2W] [0] and [S] stories. That interval measures how long the DF2W programme took, not how fast tickets turn around. The 5 issues in that sample that were created normally have a median of 0 days. BLCS ’s 9 days is the same effect: nine Stories created on 2026-06-01 and closed together on release. Both Bravo projects loaded DF2W backlog that day — it is the programme kickoff, seen twice.

3 • THE SAME TWO-HUMPED SHAPE ON BOTH

A mass of sub-tasks closing in under three days, and a tail of Stories that close on release. That is one delivery process running on two platforms, which is exactly what §2 predicted.

Limits. Each sample covers one to six working days. It is a snapshot of two release cycles, not a distribution over the year. The windows differ in length because the projects create tickets at different rates, and LN is the fastest of the three. The June LN sample is a backlog load and is not a working rhythm at all; the August windows are the ones to compare. Every sample leaves out unresolved issues, which cuts the tail short equally on all of them.

Team size is a decision already taken, not a property of the platform

THIS SECTION USED TO MAKE AN ARGUMENT. WE ARE WITHDRAWING IT.

Earlier versions read “*Seven engineers is the whole Bravo LOS delivery capacity*”, and called it **the strongest argument against putting new product families on Bravo**. That is wrong. **Bravo’s headcount is the result of an earlier CTO decision to consolidate engineering onto LORA** — not a property of Bravo, BPMN or Camunda. The office that decides where a new product family goes also sets the headcount, and can change it. **Using the consequence of a decision as evidence about that decision is circular.**

THE MEASUREMENT STILL STANDS. IT IS THE CONCLUSION DRAWN FROM IT THAT DOES NOT.

DISTINCT ASSIGNEES, 100 SAMPLED ISSUES	BRAVO BLCS	BRAVO LN	LORA BL
Across both samples	7	10	22
June sample	6	—	19
August sample	7	8	13
Resolved issues with no assignee	1	0	3
Git authors, all 4 Bravo LOS repos, 12 months		25	—

THE NUMBER WAS WRONG AS WELL AS CIRCULAR

The 7 counted **BLCS** only. Sampling **LN** the same way finds **10 more people, none of them the same**, and git history across Bravo’s four LOS repositories over 12 months shows **25 people**. The figure used against Bravo was **too low by a factor of three** — which makes withdrawing it easier, not harder.

AND IT TAKES §3 DOWN WITH IT

§3 has now been overtaken twice. It explained LORA’s **1.83×** lead as work being split more finely; headcount was a second explanation pointing the same way. Then the 1.83× itself turned out to be an artefact of the missing **LN** project, leaving **Bravo ahead at 1.27×**. The instruction is unchanged and now applies in Bravo’s favour: no per-team speed claim built on this measures the platform. **§6 survives**, because it counts pieces of work per *change* rather than tickets per team.

“The team now knows what to do” is still explained by **how long they have been there**, not by how many of them there are.

Bravo’s assignees have been on this codebase for up to four years. LORA’s include people who joined this year and are closing tickets at the same median. Knowing what to do is a property of people and their tenure, **and people can be moved**. It is not a property of either platform.

WHAT SURVIVES, AND IT FAVOURS NEITHER PLATFORM

In both samples, almost every **Story** is assigned to a **single account**, while sub-tasks spread across everyone else — 9 of 9 long-lived Stories in June, 9 of 11 in August. Whoever holds the shape of a Bravo change today is a **bus factor of one**: if that person is unavailable, the work stops — on the service carrying about 94% of origination. That is a live operational risk needing an owner **whatever is decided about new products**, and it argues for neither platform. Whether LORA’s wider spread would survive a re-staffing is unknown.

AND ONE ROW GAINS WEIGHT: WHO YOU CAN HIRE

If capacity is going to be added, the question becomes *which platform can BFI actually hire for?* Java 17, Spring Boot and Camunda 7 are common skills. Go plus a bespoke GSM planner plus Temporal is not — LORA’s own people.md puts it as “*you can get Java or Golang developers, but they won’t be able to immediately read the code.*”

This is the one staffing-related fact the CTO office cannot reverse: it is a property of the labour market, not of a headcount decision — and it favours Bravo. §7 bounds it: the advantage comes all at the start and is largely used up by week three.

The shape of one change — this is what “easier” actually means

This is the finding that survives every caveat, and the one the claim is really about. Take a single ordinary change: **upgrading an upstream integration to v2**. In LORA it appears as six tickets across five repositories, all created within two minutes on 2026-08-11, all assigned to one engineer.

BL-9528	[LSS]	add one-obligor-summary-check-v2 schema
BL-9529	[LGS]	add one-obligor-summary-check-v2 handler
BL-9530	[LPW]	migrate get-obligor-summary to v2
BL-9531	[LTS]	register v2 proxy in authorization
BL-9532	[LBOFE]	migrate auto-exposure component to v2
BL-9533	[PR]	Review: one-obligor-summary-check-v2 refactor

That is the Schema-First order — LSS → LGS → LPW/LTW → LTS → LBOFE — which LORA’s own service-dependency rule requires, written out as one ticket per step. **Nobody added this as overhead. It is the architecture, made visible in the tracker.** The pattern repeats: BL-9506 , BL-9507 and BL-9508 split one fee calculation across [LSS] , [LTW] and [LBOFE] .

THE SAME CHANGE IN BRAVO IS ONE SUB-TASK, IN ONE OF ITS TWO PROJECTS

BLCS-4433	[4W Scoring]	BE Implementation – Adjustment NST Rate – RO Rate Standard Override Indicator
-----------	--------------	---

OR, WHEN THE FRONT END IS INVOLVED, TWO LANES PLUS QA

BLCS-4388	[FE]	Development – DF2W section & field visibility on Ringkasan Data
BLCS-4389	[FE]	Manual Testing – verify DF2W reduced view + 4W no-regression
BLCS-4390	[FE]	Deploy – Deploy to SIT

AND WHEN THE CHANGE IS ON THE SURVEYOR SIDE, THE SAME SHAPE IN LN

LN-4533	[DF2W]	[0] Application Tracking Regular Risk
LN-6625	[BE]	Code Review of LN-6617
LN-6626	[FE]	Code Review of LN-6621

THE ONE CASE THAT COSTS BRAVO TWO PROJECTS

BLCS-4405 “[DF2W UW] Sync Surveyor Status Underwriting Return” is a change whose whole purpose is keeping the underwriting and surveyor sides of the boundary in step. Bravo’s split lets two squads ship independently, and this is where it pays for that. Even so, that is 2 projects against LORA’s 5 repositories in a required order.

One or two repositories, against five in a fixed order

FOR ONE INTEGRATION CHANGE	BRAVO	LORA
Repositories touched	1 – <code>bravo-bpm-service</code> ; 2 when a console is involved	5 – LSS, LGS, LPW, LTS, LBOFE
Jira projects it appears in	1 (BLCS); 2 across the underwriting and surveyor boundary	1 (BL)
Deploys needed to land it	1-2	up to 5, in a required order
Jira tickets	1-3	5-6
Backward-compatibility obligation	none inside the process	you may only add, never change or remove – 8 worker versions run at once
Cost when the change is wrong	one revert	unwinding the versions in order

This is the whole defensible content of “faster and easier”, and it is worth taking seriously.

A monolith with one relational data model really is fewer moving parts per change than thirteen repositories with a schema-first ordering rule. It also explains §3 without appealing to productivity at all: LORA writes roughly 2 to 5 times as many tickets **per change** as Bravo, so once you divide that out, the two teams may well be shipping at similar rates with very different bookkeeping. **This argument matters more now that LN is counted, not less.** Raw ticket volume now favours Bravo 1.27×, and the same correction that discounted LORA’s old 1.83× lead discounts Bravo’s new one.

WHAT THE FIVE-REPOSITORY COST BUYS

Products stay isolated at the data layer, contracts are checked against a schema, and you can add an automated step with **no change to the orchestration at all** – 172 activities against 3 hard precursors.

WHAT THE ONE-REPOSITORY CHANGE PAYS

One job executor, one deployment, and a BPMN edit that **ships for every product at once.**

Neither is free. The claim is right that Bravo’s cost per change is lower. It says nothing about what that cost buys.

How hard is each platform to join?

	BRAVO	LORA
Who you can hire	Java 17, Spring Boot and Camunda 7 – common skills	Go, a bespoke GSM planner and Temporal – “you can get Java or Golang developers, but they won’t be able to immediately read the code”
Written onboarding material	none found in this analysis pack	docs/onboarding/ – 14 files, about 5,550 lines, with tooling you can actually run
A week-1 curriculum	does not exist	does not exist either – LORA’s own people.md grades this “overstated, but the entry point is inverted”
What a new hire actually has to read	498,000 lines of Java in the main service; the product identified in 5 different places; 197 setStatus call sites across 73 files; a 10,419-line surveyor assignment service. A console hire instead reads 208,000 to 307,000 lines of React and TypeScript in one of three repositories – a smaller and far more conventional job	About 617,000 lines of Go across 13 repositories; 172 activity constructors; 155 preconditions, with no generated index of them
Ideas with no equivalent anywhere else	BPMN escalation used as a return channel; a per-application on/off matrix in jsonb; the product identified in five separate places	ReadSet and WriteSet planning; determinism constraints; schema-versioned task queues

The advantage of common skills is real, and it all comes **at the start**. It gets a new hire to their first successful compile faster. It does not help with the parts of Bravo that are genuinely hard, and on the evidence of this pack those parts are large: a central data object with no behaviour of its own, a loan lifecycle spread across four different status vocabularies, and orchestration that none of the 2,286 test files in the estate tests.

Neither platform has a week-1 curriculum. That is the finding both teams share — and it is cheaper to fix than either architecture.

What the claim gets right, and what it does not

“Developing in Lora is hard”	● PARTLY TRUE	And true in a <i>specific</i> way: five repositories, a required order to work in, schema changes that may only add, and eight worker versions running at once. Measured in §6.
“Developing in Bravo is faster”	● NOT SUPPORTED	The ticket measure reversed on a scope correction — LORA ahead 1.83× became Bravo ahead 1.27× — which retires it in both directions. What is left is an identical August median and a slower June median. No <i>reliable</i> throughput measure favours either platform.
“...and easier”	● SUPPORTED	One repository, one deploy, and no ordering across services. This is the real effect, and it is worth about 3 to 5 fewer pieces of work per change.
“...because the team now knows what to do”	● TENURE EXPLAINS IT	Bravo's assignees have been on this codebase for up to four years. LORA's include people who joined this year, closing tickets at the same median. Knowing what to do is a property of the people, not of the platform — and people can be moved. Headcount is not evidence here: see §5, where that argument is withdrawn.

A FIFTH THING THE CLAIM DOES NOT SAY, AND SHOULD

The two products it cites are the two **smallest** things Bravo runs. DF4W is the only product on the unified spine, and it takes **5.5%** of applications started over 90 days. DF2W is **not live yet** — it is in UAT with a penetration test running — and has taken **zero**. If those are the reference implementations for “Bravo is easier”, then that easiness has not yet been tested at volume, on the monoliths, where **about 94% of the book actually runs** and where 2026 saw more commits than the spine did.

Six actions

4 Set the staffing allocation, and name the bus factor · S , URGENT The Story layer runs through one account , on the service carrying about 94% of origination. That is the risk finding, and it favours neither platform. The headcount is not the finding: “seven active assignees” counted BLCS only, and the real figure is 25 people across four repositories . Before any new product family is put on Bravo, somebody has to have decided the standing allocation on purpose.	1 Stop citing boards 2099 and 2877 as evidence of delivery · S Nothing assigned, nothing resolved. Either link the DF and D2W epics to their BLCS and LN children so the timeline shows real delivery, or say plainly that Bravo delivery is tracked in both projects and compare against both. Reading only one of the two is the mistake this deck made until 2026-09-10.
2 Measure changes, not tickets · S–M This has just proved itself. The ratio swung from LORA ahead 1.83× to Bravo ahead 1.27× when one Jira project was added, with no code changing at all. A measure that moves that far on a scope fix is not measuring delivery. Agree one unit — merged pull requests per product change, or lead time at Story level — and publish it for both platforms. Until then, neither team’s speed claim can be checked.	3 Attack LORA’s cost per change directly · M This is the true part of the complaint. BL-9528 to BL-9532 are mechanical and ordered. A generator that scaffolds the schema, the handler, the activity migration, the authorization entry and the front-end call from a single spec would turn six tickets into one plus a review. LORA’s own people.md designs exactly this, as “Tier 0 — a script not a prompt”. It has not been built.
5 Write the week-1 curriculum for both platforms · S EACH Neither has one. LORA has the raw material but the wrong entry point. Bravo has easy-to-hire skills and no map of a 498,000-line monolith.	6 Ask the question again when DF2W has real volume · M DF2W is the cleanest test available of “building a new product on Bravo is easy”: configured, still pre-release, zero applications, 22 epics not started. What it actually costs — from the first BLCS ticket to the first production application — is the number this debate needs and does not have.

30, 60, 90 days

Almost none of this is Bravo engineering time. It is engineering management, PMO and, for one item, the LORA squad. That matters, because \$5 sets the limit on everything else in this pack. This plan spends about **half a day** of Squad S&U’s time.

THE RULE FOR ORDERING THIS WORK

The **staffing decision comes first**, because it sets the capacity that every other document’s plan assumes.

Days 0–30

SETTLE WHAT GETS MEASURED, AND WHO DOES IT

- 4

Name the bus factor. Decide the staffing, and the standing allocation for remediation work. *Leadership · a meeting, day 1.* Done when there is a decision on the record, including what share of Squad S&U’s time is available for the other four plans.
- 1

Tidy the boards. Link the DF and D2W epics to their **BLCS** and **LN** children, or say so in the board description — DF2W work runs through both. *PMO · 1 day.*
- 3

Prototype the LORA scaffolding generator. *LORA squad · 5-day spike.* A prototype that generates the **BL-9528** to **BL-9532** shape.

Days 31–60

AGREE WHAT A CHANGE IS

- 2

Measure changes, not tickets. Agree one unit and publish it for both platforms. *EM and the LORA squad · 3 days.* Done when “Bravo shipped 40 product changes last month, LORA 44” is a sentence somebody can check — and when ticket ratios stop being quoted as speed. That means 1.83× when it favoured LORA, and 1.27× now that it favours Bravo.

Days 61–90

REMOVE THE COST, FIX ONBOARDING

- 3

Ship the scaffolding generator. *LORA squad · 15 days.* The strongest argument for Bravo gets answered on the merits, rather than rebutted.
- 5

Week-1 curriculum, Bravo — a map of the monolith and the five places the product is identified. *EM and S&U · 3 days.*
- 5

Week-1 curriculum, LORA — change the entry point from the SDK to a loan actually moving. *LORA EM · 3 days.*
- 6

Start the DF2W clock. Measure from the first **BLCS** ticket to the first production application. *PMO · 1 day.*

Deferred

WITH A TRIGGER

- 6

Ask the “is Bravo easier” question again, with real DF2W data. DF2W has **zero** applications today and 22 epics not started, so there is nothing to measure yet. *Trigger: when DF2W takes production volume, on the clock started in phase 3.*

What changes when this is done

RECOMMENDATION	EXAMPLE WHEN DONE
Delivery tracked where delivery happens	A DF4W epic on board 2099 shows its <code>BLCS</code> and <code>LN</code> children and a real start date. The timeline stops reading as nine months of inactivity.
One agreed unit of change	"Bravo shipped 40 product changes last month, LORA shipped 44" is a sentence somebody can check, instead of two ticket counts that measure different things.
A generator for the five-repository change	<code>BL-9528</code> to <code>BL-9532</code> becomes one ticket and a generated set of pull requests. The strongest argument for Bravo loses most of its force, on the merits.
Bus factor named	Putting a new product family on Bravo becomes a decision that includes "and we will hire three more engineers" – or a decision not to.
Week-1 curriculum	A Go hire reaches a moving loan in week 1 instead of the SDK. A Java hire reaches a moving loan instead of <code>SurveyorAssignmentServiceImpl</code> .
DF2W measured end to end	The claim gets a real number attached to it, on the product it was made about.

On speed, the claim is not supported. On what it is like to work in, it is right — and the fix is a generator, not a migration.